

Proyecto Final

Analisis y diseño de algoritmos I

Mateo Echeverry Correa, Ing

2026 - I

Proyecto Final

Motor Algorítmico para Procesamiento Eficiente de Datos Dispersos o Comprimidos

1. Descripción general

El proyecto final del curso de **Análisis y Diseño de Algoritmos I** consiste en el diseño e implementación de soluciones algorítmicas eficientes para procesar grandes volúmenes de datos dispersos o comprimidos.

El propósito central del proyecto es que los estudiantes construyan algoritmos capaces de leer datos desde archivos planos, procesar consultas y operaciones, y generar salidas correctas de manera eficiente, tanto en tiempo de ejecución como en uso de memoria.

Cada equipo deberá resolver **Uno de los siguientes problemas algorítmicos**:

1. Grafos dispersos para redes de conexión.
2. Procesamiento eficiente de matrices dispersas.
3. Compresión y análisis de mapas dispersos.
4. Motor de consultas sobre datos comprimidos por rangos.

El proyecto debe evidenciar la correcta aplicación de conceptos vistos durante el curso, tales como:

- diseño de algoritmos eficientes;
- análisis de complejidad temporal;
- análisis de uso de memoria;

- implementación propia de estructuras de datos;
- procesamiento de archivos planos;
- optimización de consultas;
- representación eficiente de datos dispersos o comprimidos;
- aplicación de estrategias basadas en dividir y vencer.

El sistema no debe enfocarse en interfaces gráficas ni en aspectos visuales. La evaluación se centrará en la correcta lectura de datos, el procesamiento eficiente de operaciones, la generación exacta de salidas y la justificación algorítmica de la solución implementada.

2. Modalidad del proyecto

El proyecto se realizará en equipos de entre **2 y 4 estudiantes**.

3. Entrada y salida

Cada solución debe leer los datos desde un archivo llamado:

`entrada.txt`

Y debe generar los resultados en un archivo llamado:

`salida.txt`

El formato exacto de entrada y salida dependerá del problema desarrollado. Sin embargo, en todos los casos debe cumplirse que:

- la entrada contiene un conjunto inicial de datos;
- posteriormente se presenta un conjunto de operaciones o consultas;
- las operaciones deben procesarse en el orden en que aparecen;
- las operaciones que generen resultado deben escribirse en el archivo de salida;
- el formato de salida debe ser claro, consistente y verificable.

No se permite que el programa dependa de entrada manual interactiva, menús visuales, interfaces gráficas o intervención del usuario durante la ejecución.

4. Restricciones generales

Las soluciones deben diseñarse considerando restricciones grandes. Como referencia, los programas deben poder trabajar bajo límites similares a los siguientes:

$1 \leq N \leq 1,000,000$

$1 \leq Q \leq 200,000$

coordenadas, identificadores o posiciones hasta 10^9

Donde:

- N representa la cantidad inicial de registros, nodos, aristas, valores no nulos, elementos o rangos.
- Q representa la cantidad de operaciones o consultas.
- Las coordenadas, posiciones o identificadores numéricos pueden llegar hasta 10^9 .

Estas restricciones implican que no son aceptables soluciones que dependan de construir arreglos, matrices o estructuras completas proporcionales al tamaño total del dominio cuando este sea demasiado grande.

Por ejemplo, si una matriz tiene dimensiones:

$1000000000 \times 1000000000$

no debe construirse una matriz completa en memoria.

5. Restricciones sobre el uso de librerías y estructuras nativas

El objetivo del proyecto es evaluar la capacidad de los estudiantes para implementar y utilizar estructuras de datos propias. Por tanto, no se permite resolver el núcleo del problema usando estructuras avanzadas ya implementadas por el lenguaje o por librerías externas.

Sí se permite el uso de funcionalidades básicas para:

- lectura y escritura de archivos;
- manejo básico de cadenas;
- definición de clases, structs o módulos;
- arreglos básicos;
- funciones auxiliares;
- operaciones aritméticas;
- impresión de resultados.

5.1 Restricciones puntuales

Se permite el uso de:

open
read
write
split
listas como almacenamiento básico
clases propias
funciones propias

Las listas pueden usarse como arreglos base, pero no deben reemplazar la implementación conceptual de estructuras más complejas.

No se permite usar directamente como solución principal:

dict
set
collections.deque
heapq
bisect
queue
defaultdict
Counter
sorted
list.sort

Si el equipo requiere tablas hash, conjuntos, colas, pilas, heaps, ordenamiento eficiente o búsqueda binaria, deberá implementarlos.

6. Requerimiento de estructuras de datos

Cada equipo debe implementar estructuras de datos propias y justificar su uso dentro del documento técnico.

Dependiendo del problema seleccionado, las soluciones pueden requerir estructuras como:

- arreglos dinámicos;
- listas enlazadas;
- pilas;
- colas;
- tablas hash;
- árboles binarios;

- Montículos;
- estructuras para intervalos;
- estructuras auxiliares para recorridos;
- estructuras para ranking o selección de elementos relevantes.

El enunciado no obliga a utilizar una estructura específica en una operación específica. Sin embargo, la calificación tendrá en cuenta que las estructuras seleccionadas sean coherentes con el problema, eficientes y correctamente implementadas.

7. Requerimiento de dividir y vencer

Cada equipo debe implementar al menos **una funcionalidad relevante** usando una estrategia basada en **dividir y vencer**.

El uso de dividir y vencer debe estar integrado a una operación importante del proyecto. No se aceptará una implementación aislada sin relación con el problema principal.

Se valorará positivamente que más de una operación haga uso de este enfoque, especialmente en operaciones como:

- selección de los k elementos más relevantes;
- ordenamiento eficiente;
- búsqueda sobre datos ordenados;
- procesamiento de rangos;
- consultas por intervalos;

El documento técnico debe explicar:

- qué operación utiliza dividir y vencer;
- cuál es el problema que resuelve;
- cuál es el caso base;
- cómo se divide el problema;
- cómo se combinan o aprovechan los resultados parciales;
- cuál es su complejidad temporal;
- por qué es mejor que una solución ingenua?.

Problema 1 : Procesador de grafos dispersos para redes de conexión

1. Descripción

Dada una red de conexiones entre nodos, el programa debe construir una representación eficiente del grafo y responder consultas sobre conectividad, vecinos, grados, caminos, componentes y rankings.

La red puede representar conexiones entre ciudades, usuarios, estaciones, tareas, servidores, páginas web o cualquier otro conjunto de entidades relacionadas.

El grafo puede tener una gran cantidad de nodos, pero no necesariamente todos los nodos estarán conectados entre sí. Por esta razón, la solución debe evitar representaciones que desperdicien memoria cuando la red sea dispersa.

2. Formato sugerido de entrada

```
N M
origen destino peso
origen destino peso
...
Q
OPERACION parametros
OPERACION parametros
...
```

Donde:

- N es la cantidad de nodos.
- M es la cantidad de conexiones.
- Cada conexión tiene un nodo de origen, un nodo de destino y un peso.
- Q es la cantidad de operaciones.
- Cada operación puede requerir uno o varios parámetros.

3. Operaciones mínimas sugeridas

El equipo debe implementar operaciones que permitan consultar y analizar la red. Como mínimo, se deben considerar operaciones del siguiente tipo:

```
VECINOS nodo
GRADO nodo
EXISTE_RUTA origen destino
```

COMPONENTES
CAMINO_CORTO origen destino
NODOS_MAS_CONECTADOS k
AGREGAR_ARISTA origen destino peso
ELIMINAR_ARISTA origen destino

El equipo puede incluir operaciones adicionales si desea ampliar el alcance de su solución.

4. Ejemplo de entrada

```
8 9
A B 4
A C 2
B D 5
C D 1
D E 3
E F 2
F G 6
G H 1
C H 20
6
VECINOS A
GRADO C
EXISTE_RUTA A H
CAMINO_CORTO A H
COMPONENTES
NODOS_MAS_CONECTADOS 3
```

5. Ejemplo de salida

```
VECINOS A = B C
GRADO C = 3
EXISTE_RUTA A H = SI
CAMINO_CORTO A H = A C D E F G H
COMPONENTES = 1
NODOS_MAS_CONECTADOS 3 = C A D
```

6. Consideraciones de eficiencia

No se debe asumir que el grafo es pequeño.

Una representación basada en una matriz completa de adyacencia puede ser inviable si N es grande. El equipo debe seleccionar una representación adecuada para grafos dispersos.

Las operaciones de búsqueda, recorrido, conectividad y ranking deben implementarse considerando su complejidad temporal y espacial.

Problema 2: Procesador eficiente de matrices dispersas

1. Descripción

Dada una matriz de dimensiones potencialmente muy grandes, pero con pocos valores relevantes, el programa debe procesar operaciones sin almacenar la matriz completa.

La matriz puede tener dimensiones extremadamente grandes, pero solo una cantidad limitada de posiciones contendrá valores diferentes de cero o valores significativos.

2. Formato sugerido de entrada

```
F C N
fila columna valor
fila columna valor
...
Q
OPERACION parametros
OPERACION parametros
...
```

Donde:

- F es la cantidad total de filas.
- C es la cantidad total de columnas.
- N es la cantidad de valores no nulos iniciales.
- Luego aparecen N líneas con coordenadas y valor.
- Q representa la cantidad de operaciones.

3. Operaciones mínimas sugeridas

```
GET fila columna
SET fila columna valor
DELETE fila columna
ROW_SUM fila
COL_SUM columna
REGION_SUM f1 c1 f2 c2
TRANSPOSE
TOP_K k
DENSITY
```


4. Ejemplo de entrada

```
1000000000 1000000000 6
1 1 5
1 100 7
500 300 9
1000 1000 2
200000 10 11
999999999 999999999 15
7
GET 500 300
ROW_SUM 1
COL_SUM 100
SET 1 100 20
GET 1 100
REGION_SUM 1 1 1000 1000
TOP_K 3
```

5. Ejemplo de salida

```
GET 500 300 = 9
ROW_SUM 1 = 12
COL_SUM 100 = 7
SET 1 100 = OK
GET 1 100 = 20
REGION_SUM 1 1 1000 1000 = 36
TOP_K 3 = (1,100,20) (999999999,999999999,15) (200000,10,11)
```

6. Consideraciones de eficiencia

No se permite construir una matriz completa.

Las consultas por fila, columna y región deben resolverse usando una representación eficiente. Se valorará el uso de índices auxiliares propios si estos reducen el costo de las consultas.

Problema 3: Compresor y analizador de mapas dispersos

1. Descripción

Dado un mapa de dimensiones muy grandes, donde solo algunas coordenadas contienen elementos relevantes, el programa debe almacenar y consultar únicamente las posiciones significativas.

El mapa puede representar un terreno, una ciudad, un videojuego, una zona de riesgo, una red logística o cualquier escenario donde la mayoría de posiciones se encuentren vacías.

2. Formato sugerido de entrada

```
F C N
fila columna tipo
fila columna tipo
...
Q
OPERACION parametros
OPERACION parametros
...
```

Donde:

- F es la cantidad total de filas del mapa.
- C es la cantidad total de columnas del mapa.
- N es la cantidad de posiciones ocupadas.
- Cada posición tiene una fila, columna y tipo de elemento.
- Q representa la cantidad de operaciones.

3. Operaciones mínimas sugeridas

```
GET fila columna
SET fila columna tipo
DELETE fila columna
COUNT_TYPE tipo
REGION_COUNT f1 c1 f2 c2
K_CERCANOS fila columna k
COMPONENTES tipo
TIPOS_REGION f1 c1 f2 c2
```

4. Ejemplo de entrada

```
1000000000 1000000000 8
10 20 OBSTACULO
10 21 OBSTACULO
10 22 OBSTACULO
500 800 RECURSO
501 800 RECURSO
9000 1000 ENEMIGO
30000 40000 ESTACION
999999999 999999999 OBSTACULO
6
COUNT_TYPE RECURSO
GET 10 21
REGION_COUNT 1 1 100 100
K_CERCANOS 500 800 2
DELETE 10 22
COUNT_TYPE OBSTACULO
```

5. Ejemplo de salida

```
COUNT_TYPE RECURSO = 2
GET 10 21 = OBSTACULO
REGION_COUNT 1 1 100 100 = 3
K_CERCANOS 500 800 2 = (500,800) (501,800)
DELETE 10 22 = OK
COUNT_TYPE OBSTACULO = 3
```

6. Consideraciones de eficiencia

No se permite recorrer todo el mapa.

La solución debe operar únicamente sobre las posiciones relevantes. Las consultas espaciales deben diseñarse evitando recorridos innecesarios sobre regiones vacías.

Problema 4: Motor de consultas sobre datos comprimidos por rangos

1. Descripción

Dada una secuencia extensa representada mediante rangos comprimidos, el programa debe responder consultas y actualizaciones sin expandir completamente la secuencia.

Por ejemplo, una secuencia como:

1 1 1 1 1 3 3 3 2 2 2 2

podría representarse como:

1 aparece de la posición 1 a la 5

3 aparece de la posición 6 a la 8

2 aparece de la posición 9 a la 12

El objetivo es trabajar directamente sobre la representación comprimida.

2. Formato sugerido de entrada

```
N
valor inicio fin
valor inicio fin
...
Q
OPERACION parametros
OPERACION parametros
...
```

Donde:

- N representa la cantidad de rangos iniciales.
- Cada rango tiene un valor, una posición inicial y una posición final.
- Q representa la cantidad de operaciones.

3. Operaciones mínimas sugeridas

```
VALUE posicion
SUM inicio fin
UPDATE inicio fin valor
FREQUENCY valor
MAX_RANGE inicio fin
MIN_RANGE inicio fin
DECOMPRESS inicio fin
COUNT_RANGES
MERGE
```

4. Ejemplo de entrada

```
5
1 1 5
3 6 8
2 9 14
8 15 16
4 17 20
7
VALUE 10
SUM 1 10
UPDATE 6 8 5
FREQUENCY 2
MAX_RANGE 1 20
DECOMPRESS 1 12
COUNT_RANGES
```

5. Ejemplo de salida

```
VALUE 10 = 2
SUM 1 10 = 21
UPDATE 6 8 5 = OK
FREQUENCY 2 = 6
MAX_RANGE 1 20 = 8
DECOMPRESS 1 12 = 1 1 1 1 1 5 5 5 2 2 2 2
COUNT_RANGES = 5
```

6. Consideraciones de eficiencia

No se debe expandir toda la secuencia cuando el tamaño total del dominio sea grande.

Las operaciones sobre intervalos deben realizarse directamente sobre la representación comprimida. Se valorará la capacidad del equipo para dividir, ajustar, fusionar y consultar rangos eficientemente.

Casos límite que deben considerar

Los equipos deben probar sus soluciones con casos pequeños, medianos y grandes. Los siguientes casos deben ser considerados durante el diseño y las pruebas.

1. Para grafos dispersos

- Nodos sin conexiones.
- Grafos con múltiples componentes.
- Grafos con ciclos.
- Caminos inexistentes.
- Nodos con grado cero.
- Nodos altamente conectados.
- Conexiones repetidas.
- Pesos grandes.
- Consultas sobre nodos inexistentes.

2. Para matrices dispersas

- Matriz sin valores no nulos.
- Actualizar una posición existente.
- Actualizar una posición vacía.
- Eliminar una posición inexistente.
- Consultar una celda vacía.
- Regiones sin valores.
- Regiones que cubren toda la matriz.
- Valores negativos.
- Coordenadas grandes.

3. Para mapas dispersos

- Mapa sin elementos.
- Varios elementos del mismo tipo.
- Consultas sobre posiciones vacías.
- Regiones sin elementos.
- Regiones con muchos elementos.
- Actualización de tipos.
- Eliminación de elementos.
- Coordenadas extremas.

4. Para datos comprimidos por rangos

- Rangos consecutivos con el mismo valor.
- Actualizaciones que dividen rangos.
- Actualizaciones que fusionan rangos.
- Consultas sobre una sola posición.
- Consultas que cruzan varios rangos.
- Descompresión parcial pequeña.
- Rangos muy grandes.
- Secuencia con un único rango.

Entregables

Cada equipo debe entregar los siguientes elementos:

1. Código fuente completo.
2. Archivo `entrada.txt` con casos de prueba.
3. Archivo `salida.txt` generado por el programa.
4. Documento técnico del proyecto.
5. Instrucciones de compilación y ejecución.
6. Análisis de complejidad de operaciones principales.
7. Explicación de las estructuras de datos implementadas.
8. Explicación de la estrategia basada en dividir y vencer.

Documento técnico

El documento técnico debe contener como mínimo:

1. Portada.
2. Integrantes del equipo.
3. Descripción general del problema resuelto.
4. Formato de entrada y salida.
5. Diseño general de la solución.
6. Estructuras de datos implementadas.
7. Justificación de cada estructura.
8. Operaciones soportadas.
9. Análisis de complejidad temporal.
10. Análisis de uso de memoria.
11. Estrategia de dividir y vencer implementada.
12. Comparación con una solución ingenua.
13. Casos de prueba utilizados.
14. Casos límite considerados.
15. Conclusiones.

Análisis de estructuras de datos

Para cada estructura implementada, el equipo debe presentar una tabla como la siguiente:

Estructura implementada	Problema donde se usa	Operaciones principales	Justificación	Complejidad

Análisis de complejidad

El equipo debe analizar la complejidad de las operaciones principales de cada problema. Se recomienda utilizar una tabla como la siguiente:

Operación	Complejidad esperada	Justificación
Insertar dato		
Consultar dato		
Eliminar dato		
Procesar consulta por rango		
Calcular top-k		
Recorrer grafo		
Buscar camino		
Aplicar dividir y vencer		

El análisis debe expresarse usando notación Big O y debe estar relacionado con las variables del problema. Por ejemplo:

- N : cantidad de registros iniciales.
- Q : cantidad de consultas.
- M : cantidad de conexiones.
- K : cantidad de elementos solicitados.
- R : cantidad de rangos.

Criterios de calificación

La calificación del proyecto se realizará con base en los siguientes criterios:

Criterio	Porcentaje
Correcta lectura de <code>entrada.txt</code> y escritura de <code>salida.txt</code>	10 %
Correctitud funcional de las salidas generadas	20 %
Implementación propia y adecuada de estructuras de datos	20 %
Eficiencia temporal de las operaciones principales	15 %
Optimización del uso de memoria	10 %
Implementación de estrategia basada en dividir y vencer	10 %
Análisis de complejidad y comparación con una solución ingenua	10 %
Documento técnico, claridad y evidencias de prueba	5 %
Total	100 %

Condiciones de no aceptación o penalización

El proyecto podrá recibir una penalización significativa si ocurre alguna de las siguientes situaciones:

- El programa no compila o no ejecuta.
- No se entrega el archivo `entrada.txt`.
- No se genera el archivo `salida.txt`.
- La solución depende de una interfaz gráfica.
- Se utilizan estructuras nativas prohibidas para resolver el núcleo del problema.
- Se usan librerías externas para resolver búsquedas, ordenamientos, grafos, tablas hash, colas, pilas, árboles o rankings.
- Se construyen matrices completas o estructuras proporcionales al tamaño total del dominio cuando este puede ser de hasta 10^9 .
- No se evidencia implementación propia de estructuras de datos.
- No se presenta análisis de complejidad.
- No se implementa ninguna estrategia basada en dividir y vencer.
- El documento técnico no corresponde con el código entregado.

Consideraciones finales

Los ejemplos de entrada y salida presentados en este enunciado son ilustrativos. El docente podrá suministrar archivos adicionales con casos pequeños, medianos, grandes y casos límite para validar la correctitud y eficiencia de la solución.

Más allá de obtener una salida correcta, se espera que el equipo pueda justificar sus decisiones de diseño, explicar la complejidad de sus operaciones principales y demostrar que la solución propuesta es adecuada para procesar datos dispersos o comprimidos bajo restricciones grandes.

Recuerde que: La nota del proyecto es individual y estará determinada por un factor entre 0 y 1 que multiplicará el valor obtenido en la calificación del proyecto.